

# LAPORAN PRAKTIKUM

## STRUKTUR DATA

### SORTING 2

Disusun Oleh :

Nama : Nabila Khairunnisa  
Nim : 2511531003  
Dosen Pengampu : Dr. Wahyudi, S.T, M.T  
Asisten Praktikum : M. Galid Avero



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

TAHUN 2026

## **KATA PENGANTAR**

Puji Syukur atas kehadiran Allah SWT yang telah melimpahkan Rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum struktur data dengan judul “Sorting 2” dengan baik dan tepat waktu. Dalam menyelesaikan laporan ini saya banyak mendapat arahan dan bimbingan, oleh karena itu saya ingin mengucapkan terimakasih kepada

1. Bapak Dr. Wahyudi, S.T, M.T selaku dosen pengampu
2. Uda M. Galid Averro selaku asisten labor
3. Orang tua yang senantiasa mendoakan
4. Teman teman yang selalu memotivasi

Penulis menyadari bahwa laporan ini jauh dari kata sempurna. Oleh sebab itu, penulis sangat membuka diri apabila ada yang ingin memberikan kritikan dan saran yang sifatnya membantu, penulis akan sangat senang menerima. Tujuannya agar untuk kedepannya bisa menyempurnakan laporan.

Padang, 25 Mei 2026

Nabila Khairunnisa

## DAFTAR ISI

|                                    |            |
|------------------------------------|------------|
| <b>KATA PENGANTAR.....</b>         | <b>ii</b>  |
| <b>DAFTAR ISI.....</b>             | <b>iii</b> |
| <b>BAB I PENDAHULUAN.....</b>      | <b>1</b>   |
| 1.1 Latar Belakang .....           | 1          |
| 1.2 Tujuan Praktikum.....          | 2          |
| 1.3 Manfaat Praktikum.....         | 2          |
| <b>BAB II PEMBAHASAN.....</b>      | <b>3</b>   |
| 2.1 Deskripsi Praktikum.....       | 3          |
| 2.2 Langkah Langkah Praktikum..... | 3          |
| <b>BAB III KESIMPULAN.....</b>     | <b>11</b>  |
| 3.1 Kesimpulan .....               | 11         |
| 3.2 Saran .....                    | 11         |
| <b>DAFTAR PUSTAKA .....</b>        | <b>12</b>  |
| <b>LAPORAN TUGAS PEKAN 8 .....</b> | <b>13</b>  |

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Metode Shell Sort, juga dikenal sebagai Shell Sort, adalah algoritma pengurutan yang dikembangkan oleh Donald Shell pada tahun 1959. Algoritma ini didasarkan pada gagasan membagi daftar asli menjadi subdaftar yang lebih kecil dan mengurutkannya secara independen. Lalu, gabungkan subdaftar yang telah diurutkan ini untuk memperoleh daftar akhir yang telah diurutkan. Metode Shell Sort merupakan penyempurnaan pada algoritma penyisipan langsung karena mengurangi jumlah perbandingan dan pergeseran yang diperlukan. [1] shell sort memiliki kelebihan yaitu tidak diperlukan panggilan tumpukan, implementasi yang mudah dan efisien untuk elemen dengan jarak yang lebih kecil. Selain itu, shell sort memiliki kekurangan yaitu tidak efisien untuk ukuran array yang besar dan tidak efisien untuk elemen yang tersebar luas. Kompleksitas waktu dari algoritma shell sort adalah  $O(n^2)$ . [2]

Algoritma Quick Sort adalah salah satu algoritma pengurutan yang efisien dan banyak digunakan dalam pemrograman. Metode ini menggunakan pendekatan “Pecah dan Taklukkan” untuk mengurutkan elemen-elemen dalam sebuah daftar atau array. algoritma ini memilih suatu elemen sebagai pivot dan kemudian mempartisi sisa elemen-elemen ke dalam dua kelompok yaitu elemen-elemen yang lebih kecil dari pivot, dan elemen-elemen yang lebih besar dari pivot. Setelah itu, Quick Sort secara rekursif mengurutkan kedua kelompok tersebut hingga seluruh elemen berada pada posisi yang tepat. Kelebihan dari algoritma Quick Sort yaitu memiliki efisiensi waktu yang sangat baik, memerlukan sedikit ruang tambahan untuk menyimpan, pelaksanaan parallel yang efisien dan kinerjanya yang sangat baik dalam menangani data yang sangat besar. Meskipun memiliki kelebihan yang banyak, Quick sort juga memiliki kekurangan yaitu algoritma pengurutan yang tidak stabil, kinerja Algoritma Quick Sort dapat menjadi buruk pada kasus terburuk, Kinerja Algoritma Quick Sort sangat dipengaruhi oleh pemilihan pivot yang cerdas dan tidak secara alami cocok untuk data terikat. [3]

Algoritma Merge Sort pertama kali diperkenalkan oleh John von Neumann pada tahun 1945. Algoritma ini merupakan contoh dari teknik “divide and conquer“, di mana masalah besar dipecah menjadi sub-masalah yang lebih kecil dan lebih mudah dipecahkan. Merge Sort bekerja dengan membagi array atau daftar yang ingin diurutkan menjadi dua bagian yang lebih kecil, mengurutkan masing-masing bagian dan kemudian menggabungkannya kembali menjadi satu daftar yang terurut. [4]

Algoritma Bubble Sort adalah metode pengurutan data yang bekerja dengan cara membandingkan dua elemen yang berdekatan dan menukarnya jika posisinya salah. Proses ini dilakukan berulang kali hingga seluruh data tersusun dengan urutan yang benar. Keunggulannya terletak pada kemudahannya untuk dipahami dan diimplementasikan,

tidak membutuhkan memori tambahan, cocok untuk pembelajaran dasar, dan mudah untuk *debugging*. Di balik kelebihanannya, Bubble Sort juga memiliki beberapa keterbatasan seperti kurang cepat untuk data besar, tidak efisien dibandingkan algoritma lain, banyak memiliki perbandingan yang tidak perlu, kurang cocok untuk sistem skala besar, dan memiliki kompleksitas waktu. [5]

### **1.2 Tujuan Praktikum**

1. Mampu memahami konsep dasar sorting
2. Mampu mengetahui cara kerja algoritma Shell sort, Quick sort, Merge Sort dan Bubble sort dalam mengurutkan data array.
3. Mampu menerapkan konsep sorting dalam bentuk antarmuka grafis (GUI) menggunakan java swing melalui program bubble sort GUI.

### **1.3 Manfaat Praktikum**

1. Mahasiswa mampu memahami konsep dasar sorting.
2. Mahasiswa mampu mengetahui cara kerja algoritma Shell sort, Quick sort, Merge Sort dan Bubble sort dalam mengurutkan data array.
3. Mahasiswa mampu menerapkan konsep sorting dalam bentuk antarmuka grafis (GUI) menggunakan java swing melalui program bubble sort GUI

## BAB II

### PEMBAHASAN

#### 2.1 Deskripsi Praktikum

Praktikum Struktur data pada pekan ini adalah membuat dan mengimplementasikan beberapa algoritma pengurutan data (sorting) yaitu Bubble Sort, Shell Sort, Quick Sort, dan Merge Sort menggunakan pemrograman Java dengan antarmuka grafis (GUI). Pada praktikum ini, pengguna dapat memasukkan data berupa sekumpulan angka yang kemudian akan diurutkan menggunakan algoritma yang dipilih.

#### 2.2 Langkah Langkah Praktikum

- 1) Buatlah package terlebih dahulu dengan mengklik kanan di folder PrakSD2026\_C\_2511531003/src , pilih new dan klik package. Setelah itu beri nama pada package tanpa huruf kapital, karakter khusus serta tanpa “space”. lalu “finish”.
- 2) Klik kanan pada package pekan8\_2511531003 yang sudah dibuat sebelumnya, pilih “New”, lalu pilih class. Buat nama “ShellSort\_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini digunakan untuk mengurutkan data integer dari kecil ke besar.
- 3) Tuliskan program seperti berikut.

```
1 package pekan8_2511531003;
2
3 public class ShellSort_2511531003 {
4     public static void ShellSort_2511531003 (int[] A) {
5         int n_1003 = A.length;
6         int gap_1003 = n_1003/2;
7         while (gap_1003 > 0) {
8             for (int i = gap_1003; i < n_1003; i++) {
9                 int temp = A[i];
10                int j = i;
11                while (j >= gap_1003 && A[j - gap_1003] > temp) {
12                    A[j] = A[j - gap_1003];
13                    j = j - gap_1003;
14                }
15                A[j] = temp;
16            }
17            gap_1003 = gap_1003/2 ;
18        }
19    }
20
21    public static void main(String[] args) {
22        int[] data = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
23        System.out.print ("Sebelum: ");
24        printArray(data);
25
26        ShellSort_2511531003 (data);
27
28        System.out.print ("Sesudah (Shell Sort): ");
29        printArray(data);
30    }
31
32    public static void printArray(int [] arr) {
33        for (int i : arr) System.out.print (i + " ");
34        System.out.println ();
35    }
36 }
37 }
```

Gambar 2.2.1 kode program

- *int n\_1003 = A.length* untuk menyimpan jumlah elemen array.
- *gap* adalah jarak antar elemen yang dibandingkan.
- *While (gap\_1003 > 0)* untuk perulangan selama *gap > 0*.
- *For* untuk memeriksa setiap elemen yang dipisahkan oleh jarak *gap*.
- *Int temp = A [i]* digunakan untuk penyisipan data.
- *While* pada method 11 dan 12 digunakan untuk menggeser elemen yang lebih besar.
- *A[j] = temp* untuk menempatkan elemen pada posisi yang benar.
- *gap\_1003 = gap\_1003/2* artinya *gap* diperkecil hingga menjadi 0.
- *ShellSort\_2511531003 (data)* artinya memanggil shell sort untuk mengurutkan array.
- *PrintArray* untuk menampilkan seluruh isi array ke layar.

- 4) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut.

```
<terminated> ShellSort_2511531003 [Java Application] C:\Users\lenovo\p
Sebelum: 3 10 4 6 8 9 7 2 1 5
Setelah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

Gambar 2.2.2 Output Program

- 5) Buat lah class kedua dengan nama “QuickSort\_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini digunakan untuk mengurutkan data integer dari kecil ke besar dengan menggunakan Teknik Median Of Three untuk memilih pivot.
- 6) Tuliskan program seperti berikut.

```
1 package pekan8_2511531003;
2
3 public class QuickSort_2511531003 {
4     static void swap (int [] arr, int i, int j) {
5         int temp_1003 = arr[i];
6         arr[i] = arr[j];
7         arr[j] = temp_1003;
8     }
9     //metode tambahan untuk mengatur pivot menggunakan median of three
10    static void medianOfThree (int[] arr, int low, int high) {
11        int mid = low + (high - low) / 2;
12
13        //urutkan elemen low, mid dan high
14        if (arr[low] > arr[mid]) {
15            swap (arr, low, mid);
16        }
17        if (arr[low] > arr[high]) {
18            swap (arr, low, high);
19        }
20        if (arr[mid] > arr[high]) {
21            swap (arr, mid, high);
22        }
23        swap (arr, mid, high);
24    }
25    static int partition (int [] arr, int low, int high) {
26        // panggil fungsi medianOfThree sebelum menentukan pivot
27        medianOfThree(arr, low, high);
28
29        int pivot = arr[high]; //sekarang arr[high] sudah berisi nilai median
30        int i = (low - 1);
31    }
```

```

32 for (int j = low; j <= high - 1; j++) {
33     //jika elemen saat ini lebih kecil dari atau sama dengan pivot
34     if (arr[j] < pivot) {
35         //Increment indeks elemen yang lebih kecil
36         i++;
37         swap (arr, i, j);
38     }
39 }
40 swap (arr, i + 1, high);
41 return (i + 1);
42 }
43
44 static void QuickSort_2511531003 (int [] arr, int low, int high) {
45     if (low < high) {
46         int pi = partition(arr, low, high);
47         QuickSort_2511531003(arr, low, pi - 1);
48         QuickSort_2511531003(arr, pi + 1, high);
49     }
50 }
51
52 public static void printArr (int [] arr) {
53     for (int i = 0; i < arr.length; i++) {
54         System.out.print (arr[i] + " ");
55     }
56     System.out.println ();
57 }
58
59 public static void main (String[] args) {
60     int[] arr = { 10, 7, 8, 9, 1, 5 };
61     int N_1003 = arr.length;
62     System.out.print ("Data sebelum diurutkan: ");
63     printArr(arr);
64
65     QuickSort_2511531003(arr, 0, N_1003 - 1);
66
67     System.out.print ("Data Terurut quicksort: ");
68     printArr(arr);
69 }
70 }
71

```

Gambar 2.2.3 Kode Program

- *static void swap (int [] arr, int i, int j)* berfungsi untuk menukar dua elemen dalam array.
  - *int temp\_1003 = arr[i]* untuk menggunakan variabel sementara untuk menyimpan nilai saat pertukaran.
  - *medianOfThree* berfungsi untuk menentukan pivot menggunakan metode Median Of Three.
  - *Arr[low]* artinya elemen pertama.
  - *Arr[mid]* artinya elemen tengah.
  - *Arr[high]* artinya elemen terakhir.
  - *swap (arr, mid, high)* artinya nilai median dipindahkan ke posisi terakhir (high) sehingga dapat digunakan sebagai pivot.
  - *Partition* berfungsi membagi array berdasarkan pivot.
  - *Quicksort\_2511531003* berfungsi untuk mengurutkan array secara rekursif.
  - *printArr()* untuk menampilkan isi array.
- 7) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```

<terminated> QuickSort_2511531003 [Java Application] C:\
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10

```

Gambar 2.2.4 Output program

8) Class ketiga dengan nama “MergeSort\_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini berfungsi untuk mengurutkan data integer dari kecil ke besar dengan membagi masalah menjadi bagian kecil, menyelesaikannya, lalu menggabungkannya Kembali.

9) Tuliskan program seperti berikut.

```

1 package pekan8_2511531003;
2
3 public class MergeSort_2511531003 {
4
5     void merge_1003(int arr[], int l_1003, int m_1003, int r_1003) {
6
7         // Find sizes of two subarrays to be merged
8         int n1_1003 = m_1003 - l_1003 + 1;
9         int n2_1003 = r_1003 - m_1003;
10
11         /* Create temp arrays */
12         int L_1003[] = new int[n1_1003];
13         int R_1003[] = new int[n2_1003];
14
15         /* Copy data to temp arrays */
16         for (int i_1003 = 0; i_1003 < n1_1003; ++i_1003)
17             L_1003[i_1003] = arr[l_1003 + i_1003];
18
19         for (int j_1003 = 0; j_1003 < n2_1003; ++j_1003)
20             R_1003[j_1003] = arr[m_1003 + 1 + j_1003];
21
22         int i_1003 = 0, j_1003 = 0;
23
24         // Initial index of merged subarray array
25         int k_1003 = l_1003;
26
27         while (i_1003 < n1_1003 && j_1003 < n2_1003) {
28             if (L_1003[i_1003] <= R_1003[j_1003]) {
29                 arr[k_1003] = L_1003[i_1003];
30                 i_1003++;
31             } else {
32                 arr[k_1003] = R_1003[j_1003];
33                 j_1003++;
34             }
35             k_1003++;
36         }
37
38         /* Copy remaining elements of L[] if any */
39         while (i_1003 < n1_1003) {
40             arr[k_1003] = L_1003[i_1003];
41             i_1003++;
42             k_1003++;
43         }
44
45         /* Copy remaining elements of R[] if any */
46         while (j_1003 < n2_1003) {
47             arr[k_1003] = R_1003[j_1003];
48             j_1003++;
49             k_1003++;
50         }
51     }
52
53     void sort_1003(int arr[], int l_1003, int r_1003) {
54         if (l_1003 < r_1003) {
55             // Find the middle point
56             int m_1003 = (l_1003 + r_1003) / 2;
57
58             // Sort first and second halves
59             sort_1003(arr, l_1003, m_1003);
60             sort_1003(arr, m_1003 + 1, r_1003);
61
62             // Merge the sorted halves
63             merge_1003(arr, l_1003, m_1003, r_1003);
64         }
65     }
66
67     /* A utility function to print array of size n */
68     static void printArray_1003(int arr[]) {
69         int n_1003 = arr.length;
70
71         for (int i_1003 = 0; i_1003 < n_1003; ++i_1003)
72             System.out.print(arr[i_1003] + " ");
73
74         System.out.println();
75     }
76
77     public static void main(String args[]) {
78         int arr[] = {12, 11, 13, 5, 6, 7};
79
80         System.out.print("Sebelum Terurut : ");
81         printArray_1003(arr);
82
83         MergeSort_2511531003 ob_1003 = new MergeSort_2511531003();
84         ob_1003.sort_1003(arr, 0, arr.length - 1);
85
86         System.out.print("Setelah Terurut Merge Sort : ");
87         printArray_1003(arr);
88     }
89 }
90
91 }

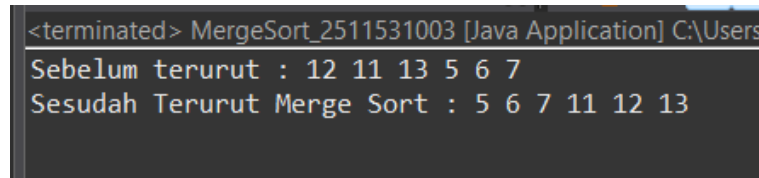
```

Gambar 2.2.5 Kode Program

- *Merge\_1003* berfungsi untuk menggabungkan dua subarray yang sudah terurut menjadi satu array yang terurut.
- *Int n1* dan *n2* untuk menentukan ukuran subarray

- *Int L dan R* digunakan untuk menyimpan bagian kiri dan kanan.
- $L\_1003[i\_1003] = arr[l\_1003 + i\_1003]$  dan  $R\_1003[j\_1003] = arr[m\_1003 + 1 + j\_1003]$  untuk menyalin data.
- *While* ( $i\_1003 < n1\_1003 \ \&\& \ j\_1003 < n2\_1003$ ) untuk menggabungkan data. Jika elemen kiri lebih kecil maka elemen kiri dimasukkan ke array utama. Jika tidak, elemen kanan dimasukkan.
- *while* ( $i\_1003 < n1\_1003$ ) digunakan untuk menyalin sisa elemen
- *sort\_1003* berfungsi untuk melakukan proses Merge sort secara rekursif.
- *if* ( $l\_1003 < r\_1003$ ) artinya jika masih lebih dari satu elemen, array akan dibagi lagi.
- *printArray\_1003* berfungsi untuk menampilkan isi array

10) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut



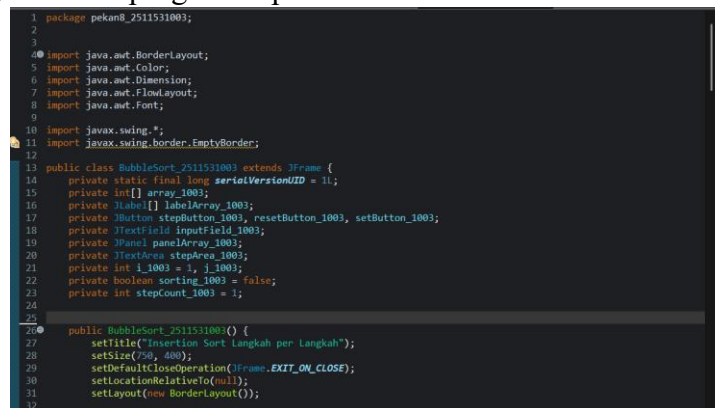
```
<terminated> MergeSort_2511531003 [Java Application] C:\Users\
Sebelum terurut : 12 11 13 5 6 7
Sesudah Terurut Merge Sort : 5 6 7 11 12 13
```

Gambar 2.2.6 Output Program

11) Class ke empat menggunakan GUI. Caranya dengan klik kanan pada package pekan7\_2511531003 kemudian arahkan kursor ke New, kemudian pilih other. Klik WindowBuilder, lalu pilih Swing Designer. Terakhir pilih JFrame dan klik tombol Next.

12) Buat nama “BubbleSort\_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi” lalu “finish”.

13) Tuliskan program seperti berikut



```
1 package pekan8_2511531003;
2
3
4 import java.net.BorderLayout;
5 import java.net.Color;
6 import java.net.Dimension;
7 import java.net.FlowLayout;
8 import java.net.Font;
9
10 import javax.swing.*;
11 import javax.swing.border.EmptyBorder;
12
13 public class BubbleSort_2511531003 extends JFrame {
14     private static final long serialVersionUID = 1L;
15     private int[] array_1003;
16     private JLabel[] labelArray_1003;
17     private JButton stepButton_1003, resetButton_1003, setButton_1003;
18     private JTextField inputField_1003;
19     private JPanel panelArray_1003;
20     private JTextArea stepArea_1003;
21     private int i_1003 = 1, j_1003;
22     private boolean sorting_1003 = false;
23     private int stepCount_1003 = 1;
24
25     public BubbleSort_2511531003() {
26         setTitle("Insertion Sort Langkah per Langkah");
27         setSize(750, 400);
28         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
29         setLocationRelativeTo(null);
30         setLayout(new BorderLayout());
31     }
32 }
```

```

33 // Panel input
34 JPanel inputPanel_1003 = new JPanel(new FlowLayout());
35 inputField_1003 = new JTextField(10);
36 setButton_1003 = new JButton("Set Array");
37 inputPanel_1003.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
38 inputPanel_1003.add(inputField_1003);
39 inputPanel_1003.add(setButton_1003);
40
41 // Panel array visual
42 panelArray_1003 = new JPanel();
43 panelArray_1003.setLayout(new FlowLayout());
44
45 // Panel kontrol
46 JPanel controlPanel_1003 = new JPanel();
47 stepButton_1003 = new JButton("Langkah Selanjutnya");
48 resetButton_1003 = new JButton("Reset");
49 stepButton_1003.setEnabled(false);
50 controlPanel_1003.add(stepButton_1003);
51 controlPanel_1003.add(resetButton_1003);
52
53 // Area teks untuk log langkah-langkah
54 stepArea_1003 = new JTextArea(8, 60);
55 stepArea_1003.setEditable(false);
56 stepArea_1003.setFont(new Font("Monospaced", Font.PLAIN, 14));
57 JScrollPane scrollPane_1003 = new JScrollPane(stepArea_1003);
58
59 // Tambahkan panel ke frame
60 add(inputPanel_1003, BorderLayout.NORTH);
61 add(panelArray_1003, BorderLayout.CENTER);
62 add(controlPanel_1003, BorderLayout.SOUTH);
63 add(scrollPane_1003, BorderLayout.EAST);
64
65 // Event Set Array
66 setButton_1003.addActionListener(e -> setArrayFromInput_1003());
67
68 // Event Langkah Selanjutnya
69 stepButton_1003.addActionListener(e -> performStep_1003());
70
71 // Event reset
72 resetButton_1003.addActionListener(e -> reset_1003());
73 }
74
75 private void setArrayFromInput_1003() {
76     String text_1003 = inputField_1003.getText().trim();
77     if (text_1003.isEmpty())
78         return;
79     String[] parts_1003 = text_1003.split(",");
80     array_1003 = new int[parts_1003.length];
81     try {
82         for (int k_1003 = 0; k_1003 < parts_1003.length; k_1003++) {
83             array_1003[k_1003] =
84                 Integer.parseInt(parts_1003[k_1003].trim());
85         }
86     } catch (NumberFormatException e_1003) {
87         JOptionPane.showMessageDialog(
88             this,
89             "Masukkan hanya angka "
90             + "yang dipisahkan koma!",
91             "Error",
92             JOptionPane.ERROR_MESSAGE
93         );
94     }
95     return;
96 }
97
98 i_1003 = 0;
99 j_1003 = 0;
100 stepCount_1003 = 1;
101 sorting_1003 = true;
102
103 stepButton_1003.setEnabled(true);
104 stepArea_1003.setText("");
105 panelArray_1003.removeAll();
106 labelArray_1003 = new JLabel[array_1003.length];
107
108 for (int k_1003 = 0; k_1003 < array_1003.length; k_1003++) {
109     labelArray_1003[k_1003] = new JLabel(String.valueOf(array_1003[k_1003]));
110     labelArray_1003[k_1003].setFont(new Font("Ariel", Font.BOLD, 24));
111     labelArray_1003[k_1003].setOpaque(true);
112     labelArray_1003[k_1003].setBackground(Color.WHITE);
113     labelArray_1003[k_1003].setBorder(BorderFactory.createLineBorder(Color.BLACK));
114
115     labelArray_1003[k_1003].setPreferredSize(new Dimension(50, 50));
116     labelArray_1003[k_1003].setHorizontalAlignment(SwingConstants.CENTER);
117     panelArray_1003.add(labelArray_1003[k_1003]);
118 }
119
120 panelArray_1003.revalidate();
121 panelArray_1003.repaint();
122
123 private void performStep_1003() {
124     if (!sorting_1003 || i_1003 >= array_1003.length - 1) {
125         sorting_1003 = false;
126         stepButton_1003.setEnabled(false);
127         JOptionPane.showMessageDialog(
128             this,
129             "Sorting selesai!"
130         );
131         return;
132     }
133
134     resetHighlights_1003();
135     StringBuilder stepLog_1003 = new StringBuilder();
136     labelArray_1003[j_1003].setBackground(Color.CYAN);
137     labelArray_1003[j_1003 + 1].setBackground(Color.CYAN);
138     if (array_1003[j_1003] > array_1003[j_1003 + 1]) {
139         // Swap
140         int temp_1003 = array_1003[j_1003];
141         array_1003[j_1003] = array_1003[j_1003 + 1];
142         array_1003[j_1003 + 1] = temp_1003;
143
144         labelArray_1003[j_1003].setBackground(Color.RED);
145         labelArray_1003[j_1003 + 1].setBackground(Color.RED);
146         stepLog_1003.append("Langkah ").append(stepCount_1003).append(": Menukar elemen ke-")
147             .append(j_1003).append(" ")
148             .append("(").append(array_1003[j_1003 + 1]).append(") dengan ke-")
149             .append(j_1003 + 1).append(" ")
150             .append("(").append(array_1003[j_1003]).append(")\n");
151     } else {
152         stepLog_1003.append("Langkah ")
153             .append(stepCount_1003).append(": Tidak ada pertukaran antara ke-")
154             .append(j_1003).append(" ")
155             .append("dan ke-")
156             .append(j_1003 + 1).append(")\n");
157     }
158
159     stepLog_1003.append("Hasil: ").append(arrayToString_1003(array_1003)).append("\n\n");
160     stepArea_1003.append(stepLog_1003.toString());
161     updateLabels_1003();
162     j_1003++;
163
164     if (i_1003 >= array_1003.length - 1 - i_1003 - 1) {

```

```

170     j_1003 = 0;
171     i_1003++;
172 }
173
174 stepCount_1003++;
175 if (i_1003 >= array_1003.length - 1) {
176     sorting_1003 = false;
177     stepbutton_1003.setEnabled(false);
178     JOptionPane.showMessageDialog(
179         this,
180         "Sorting selesai!"
181     );
182 }
183
184 private void updateLabels_1003() {
185     for (int k_1003 = 0; k_1003 < array_1003.length; k_1003++) {
186         labelArray_1003[k_1003].setText(String.valueOf(array_1003[k_1003]));
187     }
188 }
189
190 private void resetHighlights_1003() {
191     for (JLabel label_1003 : labelArray_1003) {
192         label_1003.setBackground(color.MDTE);
193     }
194 }
195
196 private void reset_1003() {
197     inputField_1003.setText("");
198     panelArray_1003.removeAll();
199     panelArray_1003.revalidate();
200     panelArray_1003.repaint();
201     stepArea_1003.setText("");
202     stepbutton_1003.setEnabled(false);
203     sorting_1003 = false;
204 }
205
206 l_1003 = 0;
207 j_1003 = 0;
208 stepCount_1003 = 1;
209
210 private String arrayToString_1003(int[] arr_1003) {
211     StringBuilder sb_1003 = new StringBuilder();
212     for (int k_1003 = 0; k_1003 < arr_1003.length; k_1003++) {
213         sb_1003.append(arr_1003[k_1003]);
214         if (k_1003 < arr_1003.length - 1)
215             sb_1003.append(", ");
216     }
217     return sb_1003.toString();
218 }
219
220 public static void main (String [] args) {
221     SwingUtilities.invokeLater(() -> {
222         BubbleSort_2511531003 gui_1003 = new BubbleSort_2511531003 ();
223         gui_1003.setVisible(true);
224     });
225 }

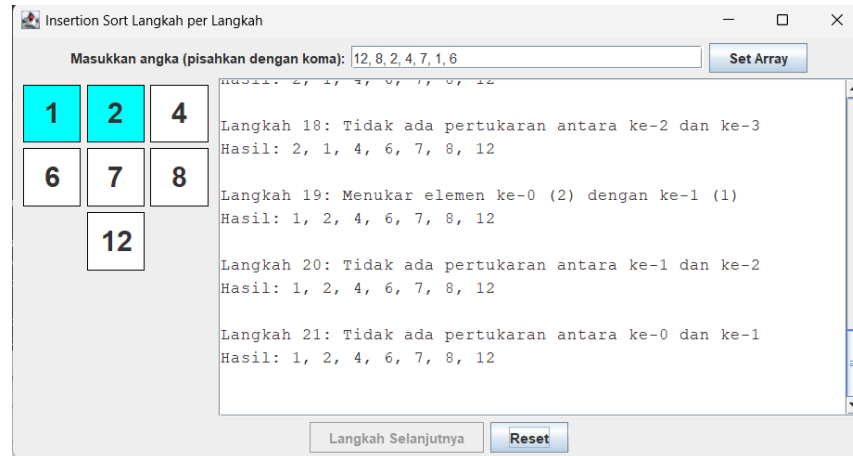
```

Gambar 2.2.7 Kode Program

- Constructor *BubleSort* digunakan untuk membuat tampilan awal aplikasi yang berfungsi untuk memberi judul window, mengatur ukuran window, membuat program berhenti saat window ditutup, menampilkan window di tengah layar, memakai layout BorderLayout.
- Panel input untuk membuat tempat input angka dan tombol set array.
- Panel array visual digunakan untuk menampilkan angka array dalam bentuk label/kotak visual.
- Panel kontrol digunakan untuk membuat tombol langkah selanjutnya untuk menjalankan sorting per langkah, reset untuk menghapus data, tombol langkah awalnya dimatikan karena array belum dimasukkan.
- Area teks untuk log langkah langkah digunakan untuk menampilkan catatan proses sorting
- Tambahkan panel ke frame digunakan untuk menambahkan panel input ke bagian atas frame.
- Event tombol artinya jika tombol *Set Array* di klik, program akan menjalankan *setArrayFromInput()*. Jika tombol *langkah selanjutnya* diklik, program akan menjalankan *performStep()*. Jika tombol *reset* diklik, program akan menjalankan *reset()*.
- Method *setArrayFromInput()* digunakan untuk mengambil angka dari input user
- Method 100-125 berfungsi untuk mengatur kondisi awal Bubble Sort.
- *performStep\_1003* dipanggil setiap kali tombol step ditekan.

- *resetHighlights\_1003* berfungsi untuk mengembalikan semua kotak menjadi putih.
- *updateLabels\_1003* digunakan untuk memperbarui angka pada JLabel.
- *Reset\_1003* digunakan saat tombol reset ditekan.
- *arrayToString\_1003* untuk mengubah array menjadi string.

14) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut



Gambar 2.2.8 Output Program

## **BAB III**

### **KESIMPULAN**

#### **3.1 Kesimpulan**

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa setiap algoritma memiliki cara kerja dan Tingkat efisiensi yang berbeda dalam proses pengurutan data. Bubble sort bekerja dengan membandingkan dua elemen yang berdekatan dan menukarnya jika urutannya salah. Shell sort merupakan pengembangan dari insertion sort yang menggunakan jarak (gap) tertentu untuk mempercepat proses pengurutan. Quick sort menggunakan metode dengan memilih pivot untuk membagi data menjadi dua bagian. Merge sort menggunakan metode dengan membagi beberapa bagian kecil, kemudian menggabungkannya kembali dalam keadaan terurut.

#### **3.2 Saran**

Untuk menambah pengembangan praktikum di masa mendatang disarankan untuk tampilan GUI dapat dibuat lebih menarik dan responsif dengan penambahan warna, grafik atau statistic yang menunjukkan jumlah perbandingan dan pertukaran yang terjadi selama proses sorting.

## DAFTAR PUSTAKA

- [1] T. Digital, "Metode Sortir Shell dalam C dan Java: Panduan Lengkap," InformatecDigital, 19 Oktober 2025. [Online]. Available: <https://informatecdigital.com/id/Metode-sortir-shell-di-cy-java-panduan-lengkap/>. [Accessed 31 Mei 2026].
- [2] S. Chen, "Algoritma Pengurutan Shell dengan CONTOH," Guru99, 28 Juli 2025. [Online]. Available: <https://www.guru99.com/id/shell-sort-algorithm.html>. [Accessed 31 Mei 2026].
- [3] M. M. Sutiono S.Kom., "Algoritma Quick Sort: Pengertian, Kelebihan dan Contoh," DosenIT.com, 23 Agustus 2024. [Online]. Available: <https://dosenit.com/kuliah-it/algoritma-quick-sort>. [Accessed 31 Mei 2026].
- [4] KantiniT, "Algoritma Merge Sort: Pengertian, Cara Kerja dan Implementasi," kantinit.com, 22 September 2023. [Online]. Available: <https://kantinit.com/programming/algoritma-merge-sort-pengertian-cara-kerja-dan-implementasi/>. [Accessed 31 Mei 2026].
- [5] I. A. Namira, "Algoritma Bubble Sort Adalah: Arti, Cara Kerja, dan Contohnya," Coding Studio, 7 April 2026. [Online]. Available: <https://codingstudio.id/blog/algoritma-bubble-sort-adalah/>. [Accessed 31 Mei 2026].

## LAPORAN TUGAS PRAKTIKUM PEKAN 8

### 1. KODE PROGRAM

- **// CLASS 1 : Lagu\_2511531003**

```
package pekan8_2511531003;
```

```
public class Lagu_2511531003 {
```

```
    String judul_1003;
```

```
    String penyanyi_1003;
```

```
    int durasi_1003;
```

```
    public Lagu_2511531003(String judul_1003, String penyanyi_1003, int durasi_1003)
    {
        this.judul_1003 = judul_1003;
        this.penyanyi_1003 = penyanyi_1003;
        this.durasi_1003 = durasi_1003;
    }
}
```

- **//CLASS 2 : Sorting\_2511531003**

```
package pekan8_2511531003;
```

```
import java.util.Scanner;
```

```
public class Sorting_2511531003 {
```

```
    Lagu_2511531003[] dataLagu_1003 = new Lagu_2511531003[20];
```

```
    int jumlah = 0;
```

```
    Scanner input = new Scanner(System.in);
```

```
    public void inputData_1003() {
```

```
        System.out.print("Masukkan jumlah lagu (minimal 7, maksimal 20): ");
```

```
        jumlah = input.nextInt();
```

```
        input.nextLine();
```

```
        if (jumlah < 7 || jumlah > 20) {
```

```
            System.out.println("Jumlah lagu harus antara 7 sampai 20!");
```

```

        return;
    }

    for (int i = 0; i < jumlah; i++) {

        System.out.println("\nData Lagu ke-" + (i + 1));

        System.out.print("Judul   : ");
        String judul = input.nextLine();

        System.out.print("Penyanyi : ");
        String penyanyi = input.nextLine();

        System.out.print("Durasi (detik) : ");
        int durasi = input.nextInt();
        input.nextLine();

        dataLagu_1003[i] =
            new Lagu_2511531003(judul, penyanyi, durasi);
    }
}

public void tampilData_1003() {

    for (int i = 0; i < jumlah; i++) {

        System.out.println((i + 1) + ". "
            + dataLagu_1003[i].judul_1003
            + " - "
            + dataLagu_1003[i].durasi_1003
            + " detik");
    }
}

public void quickSort_1003(int low, int high) {

    if (low < high) {

        int pivotIndex = partition(low, high);
    }
}

```

```

        quickSort_1003(low, pivotIndex - 1);
        quickSort_1003(pivotIndex + 1, high);
    }
}

private int partition(int low, int high) {
    int pivot = dataLagu_1003[high].durasi_1003;
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (dataLagu_1003[j].durasi_1003 < pivot) {
            i++;

            Lagu_2511531003 temp = dataLagu_1003[i];
            dataLagu_1003[i] = dataLagu_1003[j];
            dataLagu_1003[j] = temp;
        }
    }
    Lagu_2511531003 temp = dataLagu_1003[i + 1];
    dataLagu_1003[i + 1] = dataLagu_1003[high];
    dataLagu_1003[high] = temp;

    return i + 1;
}

public static void main(String[] args) {
    Sorting_2511531003 playlist =
        new Sorting_2511531003();

    playlist.inputData_1003();
    if (playlist.jumlah < 7 || playlist.jumlah > 20) {
        return;
    }

    System.out.println();
    System.out.println("=== Sorting Playlist NIM: 2511531003 ===");
    System.out.print("\nPilih Algoritma (1=Shell, 2=Quick, 3=Merge): ");
    int pilih = playlist.input.nextInt();

    System.out.println("\nData Sebelum Sorting:\n");

```

```

playlist.tampilData_1003();
switch (pilih) {

case 1:
    System.out.println("\nShell Sort belum diimplementasikan.");
    break;

case 2:
    playlist.quickSort_1003(0, playlist.jumlah - 1);
    System.out.println("\nData Setelah Quick Sort (Durasi Asc):\n");
    playlist.tampilData_1003();
    break;

case 3:
    System.out.println("\nMerge Sort belum diimplementasikan.");
    break;

default:
    System.out.println("\nPilihan tidak valid.");
}
}
}

```

## 2. SCREENSHOT CODE PROGRAM

- **Class 1 : Lagu\_2511531003**

```
1 package pekan8_2511531003;
2
3 public class Lagu_2511531003 {
4     String judul_1003;
5     String penyanyi_1003;
6     int durasi_1003;
7
8     public Lagu_2511531003(String judul_1003, String penyanyi_1003, int durasi_1003) {
9         this.judul_1003 = judul_1003;
10        this.penyanyi_1003 = penyanyi_1003;
11        this.durasi_1003 = durasi_1003;
12    }
13 }
14
```

- **Class 2 : Sorting\_2511531003**

```
1 package pekan8_2511531003;
2
3 import java.util.Scanner;
4
5 public class Sorting_2511531003 {
6     Lagu_2511531003[] dataLagu_1003 = new Lagu_2511531003[20];
7     int jumlah = 0;
8
9     Scanner input = new Scanner(System.in);
10    public void inputData_1003() {
11        System.out.print("Masukkan jumlah lagu (minimal 7, maksimal 20): ");
12        jumlah = input.nextInt();
13        input.nextLine();
14
15        if (jumlah < 7 || jumlah > 20) {
16            System.out.println("Jumlah lagu harus antara 7 sampai 20!");
17            return;
18        }
19
20        for (int i = 0; i < jumlah; i++) {
21
22            System.out.println("\nData Lagu ke-" + (i + 1));
23
24            System.out.print("Judul : ");
25            String judul = input.nextLine();
26
27            System.out.print("Penyanyi : ");
28            String penyanyi = input.nextLine();
29
30            System.out.print("Durasi (detik) : ");
31            int durasi = input.nextInt();
32            input.nextLine();
33
34            dataLagu_1003[i] =
35                new Lagu_2511531003(judul, penyanyi, durasi);
36        }
37    }
38
39    public void tampilData_1003() {
40
41        for (int i = 0; i < jumlah; i++) {
42
43            System.out.println((i + 1) + ". "
44                + dataLagu_1003[i].judul_1003
45                + " - "
46                + dataLagu_1003[i].durasi_1003
47                + " detik");
48        }
49    }
50
51    public void quickSort_1003(int low, int high) {
52
53        if (low < high) {
54
55            int pivotIndex = partition(low, high);
56
57            quickSort_1003(low, pivotIndex - 1);
58            quickSort_1003(pivotIndex + 1, high);
59        }
60    }
61
62    private int partition(int low, int high) {
63        int pivot = dataLagu_1003[high].durasi_1003;
64        int i = low - 1;
65
66        for (int j = low; j < high; j++) {
67            if (dataLagu_1003[j].durasi_1003 < pivot) {
68                i++;
69
70                Lagu_2511531003 temp = dataLagu_1003[i];
71                dataLagu_1003[i] = dataLagu_1003[j];
72                dataLagu_1003[j] = temp;
73            }
74        }
75    }
76}
```

```

75     lagu_2511531003 temp = dataLagu_1003[i + 1];
76     dataLagu_1003[i + 1] = dataLagu_1003[high];
77     dataLagu_1003[high] = temp;
78
79     return i + 1;
80 }
81
82 public static void main(String[] args) {
83     Sorting_2511531003 playlist =
84         new Sorting_2511531003();
85
86     playlist.inputData_1003();
87     if (playlist.jumlah < 7 || playlist.jumlah > 20) {
88         return;
89     }
90
91     System.out.println();
92     System.out.println("=== Sorting Playlist NIM: 2511531003 ===");
93     System.out.print("\nPilih Algoritma (1=Shell, 2=Quick, 3=Merge): ");
94     int pilih = playlist.input.nextInt();
95
96     System.out.println("\nData Sebelum Sorting:\n");
97     playlist.tampilData_1003();
98     switch (pilih) {
99
100        case 1:
101            System.out.println("\nShell Sort belum diimplementasikan.");
102            break;
103
104        case 2:
105            playlist.quickSort_1003(0, playlist.jumlah - 1);
106            System.out.println("\nData Setelah Quick Sort (Durasi Asc):\n");
107            playlist.tampilData_1003();
108            break;
109
110        case 3:
111            System.out.println("\nMerge Sort belum diimplementasikan.");
112            break;
113
114        default:
115            System.out.println("\nPilihan tidak valid.");
116
117    }
118 }

```

### 3. OUTPUT

```

<terminated> Sorting_2511531003 [Java Application] C:\Users\lenovo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.8.v20250724-1412\jre\bin\java
Masukkan jumlah lagu (minimal 7, maksimal 20): 10

Data Lagu ke-1
Judul   : Satu Bulan
Penyanyi : Bernadya
Durasi (detik) : 200

Data Lagu ke-2
Judul   : Penjaga Hati
Penyanyi : Nadhif Basalamah
Durasi (detik) : 260

Data Lagu ke-3
Judul   : Lantas
Penyanyi : Juicy Luicy
Durasi (detik) : 234

Data Lagu ke-4
Judul   : Bertaut
Penyanyi : Nadin Amizah
Durasi (detik) : 315

Data Lagu ke-5
Judul   : Nina
Penyanyi : .Feast
Durasi (detik) : 277

Data Lagu ke-6
Judul   : Tak Kan Hilang
Penyanyi : Budi Doremi
Durasi (detik) : 241

Data Lagu ke-7
Judul   : Pura Pura Lupa
Penyanyi : Mahen
Durasi (detik) : 246

Data Lagu ke-8
Judul   : Shape Of My Heart
Penyanyi : Backstreet Boys
Durasi (detik) : 230

Data Lagu ke-9
Judul   : That Should Be Me
Penyanyi : Justin Bieber
Durasi (detik) : 232

Data Lagu ke-10
Judul   : Wide Awake
Penyanyi : Katy Perry
Durasi (detik) : 220

```

```

=== Sorting Playlist NIM: 2511531003 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2
Data Sebelum Sorting:
1. Satu Bulan - 200 detik
2. Penjaga Hati - 260 detik
3. Lantas - 234 detik
4. Bertaut - 315 detik
5. Nina - 277 detik
6. Tak Kan Hilang - 241 detik
7. Pura Pura Lupa - 246 detik
8. Shape Of My Heart - 230 detik
9. That Should Be Me - 232 detik
10. Wide Awake - 220 detik

Data Setelah Quick Sort (Durasi Asc):
1. Satu Bulan - 200 detik
2. Wide Awake - 220 detik
3. Shape Of My Heart - 230 detik
4. That Should Be Me - 232 detik
5. Lantas - 234 detik
6. Tak Kan Hilang - 241 detik
7. Pura Pura Lupa - 246 detik
8. Penjaga Hati - 260 detik
9. Nina - 277 detik
10. Bertaut - 315 detik

```

#### 4. PENJELASAN TUGAS

Pada tugas pekan 8 ini, diperintahkan untuk mengimplementasikan salah satu algoritma sorting untuk mengurutkan data lagu dalam playlist yaitu antara Shell Sort, Quick Sort, atau Merge Sort. Data lagu disimpan dalam array (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (A Z) atau Durasi (terpendek ke terpanjang).

Pada tugas ini, saya memilih untuk mengimplementasikan algoritma sorting Quick Sort. Alasannya karena Quick Sort salah satu metode proses pengurutan yang cepat. Quick Sort bekerja dengan membagi data menjadi beberapa bagian (partition) berdasarkan sebuah pivot, kemudian mengurutkan setiap bagian secara rekursif. Sehingga algoritma ini paling efektif digunakan untuk data berukuran sedang hingga besar.

Program ini terdiri dari dua class, yaitu class Lagu\_2511531003 dan Sorting\_2511531003. Class Lagu\_2511531003 merupakan class yang digunakan untuk merepresentasikan data lagu dalam program playlist. Class ini berfungsi sebagai tempat penyimpanan atribut atribut yang dimiliki oleh setiap lagu, yaitu judul\_1003, penyanyi\_1003, dan durasi\_1003. Atribut judul\_1003 digunakan untuk menyimpan judul lagu, atribut penyanyi\_1003 digunakan untuk menyimpan nama penyanyi, sedangkan atribut durasi\_1003 digunakan untuk menyimpan lama durasi lagu dalam satuan detik. Dengan adanya class ini, pengelolaan data lagu menjadi lebih terstruktur karena setiap lagu direpresentasikan dalam satu objek yang memiliki atribut lengkap.

Class Sorting\_2511531003 merupakan class utama yang digunakan untuk mengelola data playlist lagu sekaligus menjalankan proses pengurutan (sorting). Class ini memiliki array dataLagu\_1003 yang berfungsi untuk menyimpan minimal 7 lagu dan maksimal 20 lagu dari class Lagu\_2511531003. Selain itu, class ini juga menggunakan objek Scanner untuk menerima input data lagu dari pengguna. Method inputData\_1003() berfungsi untuk menerima input jumlah lagu, judul lagu, nama penyanyi, dan durasi lagu yang kemudian disimpan ke dalam array dataLagu\_1003. Method tampilData\_1003() digunakan untuk menampilkan seluruh data lagu yang telah dimasukkan oleh pengguna baik sebelum

maupun sesudah proses sorting dilakukan. Method `quickSort_1003()` digunakan untuk mengurutkan data lagu berdasarkan durasi secara ascending (dari yang terkecil ke yang terbesar) menggunakan algoritma Quick Sort. Method `main()` berfungsi untuk menampilkan judul program, memanggil method input data, meminta pengguna memilih algoritma sorting, menampilkan data sebelum sorting, menjalankan proses Quick Sort, dan menampilkan hasil pengurutan.